

Introduction

For my final project, I decided to complete an analysis on the housing data. Upon initial reading of the dataset's description, I thought the most influential factors on MEDV would likely be CRIM, LSTAT, and AGE based purely on my guessing.

Because our analysis is being done on MEDV, which is a numeric (non-categorical) variable, I decided to go with regression rather than classification. Regression also allows us to predict something using many different variables, which we have 12 of in our housing dataset.

Data Preparation

Very little data preparation was needed because almost all of the variables were already numerical. However, one variable— RIVER— was coded as yes/no. So I used a dummy code to convert River=Yes to True/False or 1/0 (1 = True, 0 = False). Regression is highly numerical, and not converting with dummy coding could cause problems down the line with the models as they expect numerical data.

```
housing = pd.concat([housing, pd.get_dummies(housing['RIVER'], prefix='RIVER', drop_first=True)], axis=1)
housing.drop(['RIVER'], inplace=True, axis=1)
```

```
housing.head(5)
```

	CRIM	ZN	INDUS	NOX	RM	AGE	DIS	RAD	TAX	PRATIO	LSTAT	MEDV	RIVER_Yes
0	3.32105	0.0	19.58	0.871	5.403	100.0	1.3216	5	403	14.7	26.82	13.4	True
1	1.12658	0.0	19.58	0.871	5.012	88.0	1.6102	5	403	14.7	12.12	15.3	True
2	1.41385	0.0	19.58	0.871	6.129	96.0	1.7494	5	403	14.7	15.12	17.0	True
3	3.53501	0.0	19.58	0.871	6.152	82.6	1.7455	5	403	14.7	15.02	15.6	True
4	1.27346	0.0	19.58	0.605	6.250	92.6	1.7984	5	403	14.7	5.50	27.0	True

Multiple Regression Model Using OLS

Using OLS, I did an initial input of the dataset to see what the results would be. Reviewing the summary, we can see that OLS has an R-Square value of only 0.514, meaning only 51.4% of our data can be explained by the model. While this is not the worst possible fit, it is not going to give us the most direct or relevant insights. We also know that the model is statistically significant as our F-Statistic is 2.99e-71, which is far below 0.05.

According to our OLS model, all of our variables are statistically significant, as all of the $P > |t|$ values are below 0.05. We can also see exactly how our variables affect the MEDV. For example, homes by the river typically cost \$4,569.40 more than homes that are not by the river. We also see the higher crime, higher industrial land in home radius, older age, further distance from

employment centers, higher tax rates, and more students per teacher all decrease median home value.

```
print(result.summary())
```

OLS Regression Results						
=====						
Dep. Variable:	MEDV	R-squared:		0.514		
Model:	OLS	Adj. R-squared:		0.504		
Method:	Least Squares	F-statistic:		52.38		
Date:	Tue, 12 May 2026	Prob (F-statistic):		2.99e-71		
Time:	19:54:28	Log-Likelihood:		-1657.6		
No. Observations:	506	AIC:		3337.		
Df Residuals:	495	BIC:		3384.		
Df Model:	10					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

Intercept	83.7186	4.589	18.245	0.000	74.703	92.734
RIVER_Yes[T.True]	4.5694	1.168	3.911	0.000	2.274	6.865
CRIM	-0.2543	0.043	-5.845	0.000	-0.340	-0.169
ZN	0.0747	0.018	4.041	0.000	0.038	0.111
INDUS	-0.2074	0.083	-2.512	0.012	-0.370	-0.045
NOX	-32.1727	5.119	-6.285	0.000	-42.231	-22.114
AGE	-0.0387	0.017	-2.286	0.023	-0.072	-0.005
DIS	-2.2078	0.267	-8.259	0.000	-2.733	-1.683
RAD	0.4450	0.089	5.005	0.000	0.270	0.620
TAX	-0.0172	0.005	-3.370	0.001	-0.027	-0.007
PRATIO	-1.4877	0.174	-8.552	0.000	-1.830	-1.146

Train Test Split & Linear Regression

Continuing the project, I decided to employ Train Test Split and Linear Regression to create a more accurate model and be able to tell how well we can predict median home values.

The R^2 values are higher for the linear regression model than our earlier OLS model. Our training part of the model (the values we gave to it) has an R^2 of 74.7%, which means 74.7% of the median home values can be explained by our model. Additionally, testing has an R^2 of 68.4%, which is lower than the training values but still higher than the OLS model.

```
print('RMSE train: %.3f, test: %.3f' % (
    sqrt(mean_squared_error(y_train, y_train_pred)),
    sqrt(mean_squared_error(y_test, y_test_pred))))
print('R^2 train: %.3f, test: %.3f' % (
    r2_score(y_train, y_train_pred),
    r2_score(y_test, y_test_pred)))
```

```
RMSE train: 4.574, test: 5.384
R^2 train: 0.747, test: 0.684
```

Additionally, I calculated the RMSE for training and testing. RMSE for training is 4.574, which means that on average, our training results are \$4,574 off. RMSE for testing is 5.384, which means we can expect our prediction results to be about \$5,384 off. While we want the numbers to be as low as possible for accuracy's sake, these are not too terribly far off considering we are discussing home values.

I also calculated the coefficients:

```
print("Coefficients:\n", regressor.coef_)
print("Intercept:\n", regressor.intercept_)
```

```
Coefficients:
[[-1.27602565e-01  4.62298207e-02 -5.10691067e-03 -1.54465024e+01
  3.80864472e+00  7.01507805e-03 -1.39215520e+00  2.37284247e-01
 -1.08450209e-02 -9.68409524e-01 -5.46033184e-01  2.89693965e+00]]
```

Most of the coefficients stayed relatively close to the OLS regression model results. The biggest difference in NOX, which was -32.1727 (-\$32,172.70) in the OLS model and -15.4465 (-\$15,446.50) in our Test Train Split model. However, while this change may seem drastic, it is also one of the values that has a very low range, meaning a much smaller difference between the values in each row.

Predicting Values

To test if the model could predict values, I first tested with some values that were already in the dataset to see how accurate it is. Based on the aforementioned RMSE on the testing values, we can expect around \$5,384 in difference.

In my first prediction test, I tested the very first row, which had a 13.4 MEDV. The prediction was not far off, at only ~14.918. This was a \$1,518 difference between actual and predicted.

```
prediction = regressor.predict([[3.32105, 0.0, 19.58, 0.871, 5.403, 100.0, 1.3216, 5, 403, 14.7, 26.82, 1]])
print('Prediction:', prediction)

#Correct answer should be 13.4
```

```
Prediction: [[14.91790762]]
```

In my second test prediction, the actual answer was 13.2 and the predicted value was ~9.929. This was a slightly bigger gap, of about \$3,271.

```
prediction2 = regressor.predict([[1.38799, 0, 8.14, 0.538, 5.95, 82, 3.99, 4, 307, 21, 27.71, 0]])
print('Prediction:', prediction2)

#Correct answer should be 13.2
```

```
Prediction: [[9.92885827]]
```

Finally, my third prediction had the largest difference. The actual value was 35.4 and the predicted value was ~30.981, leaving a gap of \$4,419.

```
prediction3 = regressor.predict([[0.01311, 90, 1.22, 0.403, 7.249, 21.9, 8.6966, 5, 226, 17.9, 4.81, 0]])  
print('Prediction:', prediction3)
```

```
#Correct answer should be 35.4
```

```
Prediction: [[30.98106398]]
```

While the gaps do have some variation, all of the ones I tested stayed within the \$5,384 RMSE score I calculated earlier. Additionally, the variations do not seem to be only negative or positive, as the first prediction was above the actual MEDV and the last two were both below the MEDV.

Finally, I created my own prediction. I chose the values by calculating the min, max, and mean for the dataset and inputted values that were within a reasonable range of the mean. The predicted MEDV based on my values was 24.33141, or a home value of \$24,331.41. This seems pretty accurate, as the mean MEDV is \$22,532.81. The difference is \$2,798.60, which is within our predicted RMSE.

```
prediction4 = regressor.predict([[4.1, 9.84, 12.11, 0.55, 5.92, 55, 2.11, 10, 311, 20, 15, 1]])  
print('Prediction:', prediction4)
```

```
Prediction: [[24.33140948]]
```

Take Aways

Firstly, using Train Test Split (TTS) with linear regression fits the dataset much better than when using OLS for a multiple regression model. TTS also gave me the ability to predict MEDV scores and do further analysis, as it is much more comprehensive.

Next, I think that using TTS is very relevant when trying to predict MEDV. It also showed the variables that had the largest effects on MEDV. Average number of rooms (RM) and being by the river (RIVER_Yes) both increased median home value MEDV significantly, while crime rate by town, nitrogen oxide pollution (NOX), pupil-student ratio (PRATIO), and lower economic status by town (LSTAT) all decreased median home value significantly.

Looking at the coefficients, we can see exactly what effects each of these variables had on MEDV. First is RM, which had a coefficient of 3.80864. This means that a house adding one bedroom created an increased value of \$3,808.64. RIVER_Yes had a coefficient of 2.89694, meaning a house along the river was worth \$2,896.94 more than a house that was not.

NOX had a negative coefficient of -15.44650, which means every step up in NOX decreased a home's value by \$15,446.50. This may sound like a drastic change, but as stated earlier, the

range of NOX is only 0.385-0.871, meaning no home value actually has that large of a difference due to the NOX variable. Nonetheless, it still had a significant change on MEDV. PRATIO describes the average ratio of teachers to students in a town, and a bigger ratio (more students, fewer teachers) decreased a home's value by 968.41 (-0.96841 coefficient). Finally, LSTAT had a coefficient of -0.56303, which means a 1% increase in lower status population by town caused the home value to go down by \$563.03. While that may not sound like a lot, going from a 5% lower status to 15% means an average decrease of \$5,630.30 to all homes in the town. This can be pretty substantial, especially as there are typically very few exceptionally low LSTAT towns.

I was surprised to not see AGE be a more contributing factor, as it had a coefficient of only 0.007015 (or a \$7.01 increase in home value for every one year). I was expecting it to have a largely negative effect, as homes often depreciate over time. However, I was correct in my predictions that CRIM and LSTAT both would affect the MEDV.

Overall, this project has shown that it is possible to pretty accurately predict median home values based on factors like economic status, crime rates, number of rooms, and other factors. It has also shown that the best way to go about this (based on methods we were taught in class) is Train Test Split with Linear Regression. This methodology can be used in other datasets where we want to predict a numerical value, and we can use dummy coding to use non-numerical factors to predict them.